

Implementing Async RAI

WARNING: Code-heavy talk

ROBERT LEAHY



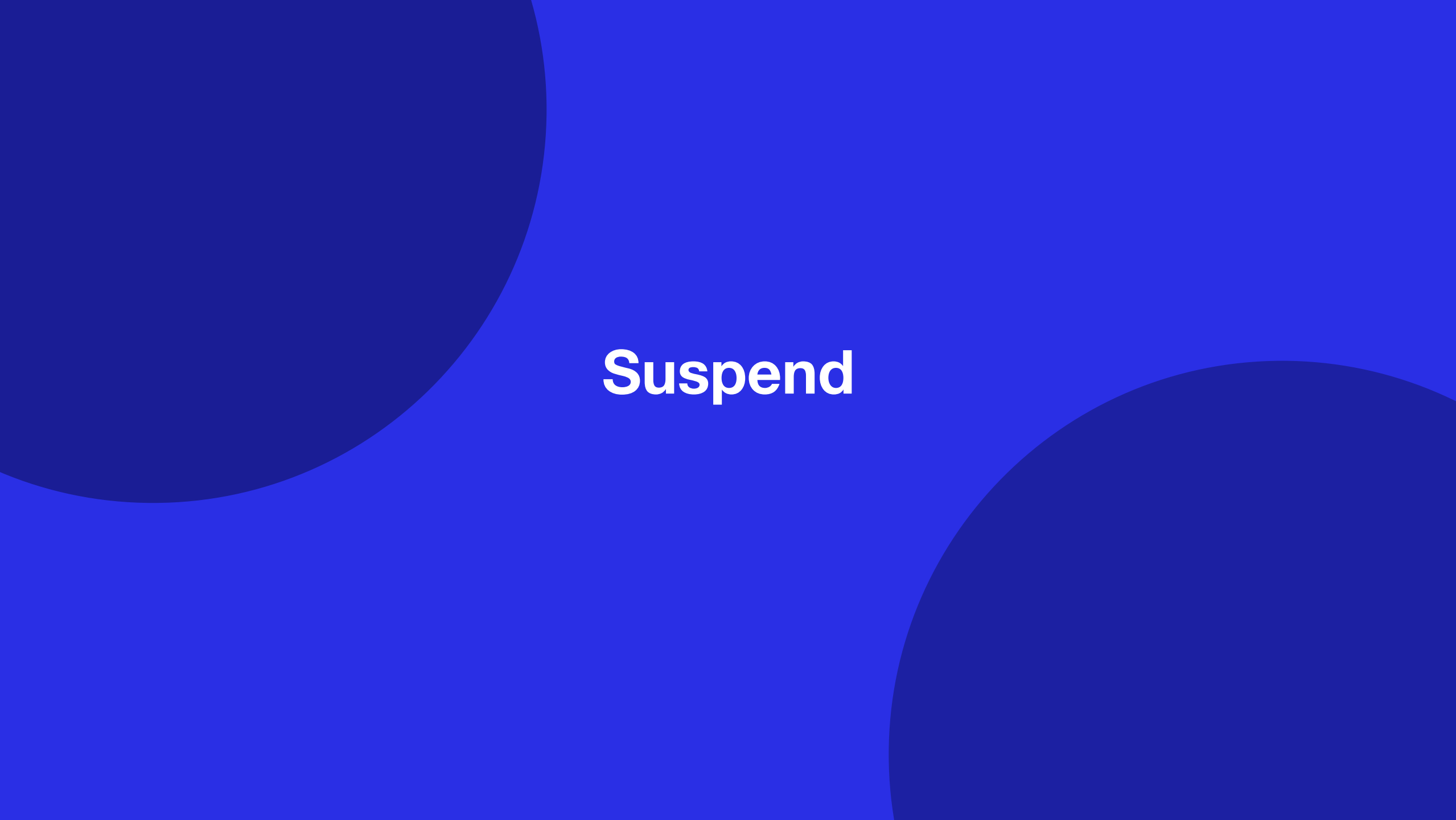
C/C++

[Everyone disliked that]

```
{  
  foo f;  
  // ...  
}
```

```
{  
  foo f;  
  // ...  
}
```

```
foo f;  
// ...
```



Suspend

Structured Concurrency

Coroutine-Based Construction & Destruction

Construction

A constructor shall not be a coroutine.

§11.4.5.1 [class.ctor.general]

Destruction

A destructor shall not be a coroutine.

§11.4.7 [class.dtor]

IORING_OP_OPENAT

Issue the equivalent of a `openat(2)` system call. `fd` is the `dirfd` argument, `addr` must contain a pointer to the `*pathname` argument, `open_flags` should contain any flags passed in, and `len` is access mode of the file. See also `openat(2)` for the general description of the related system call. Available since 5.6.

IORING_OP_CLOSE

Issue the equivalent of a `close(2)` system call. `fd` is the file descriptor to be closed. See also `close(2)` for the general description of the related system call. Available since 5.6. If the `file_index` field is set to a positive number, this command can be used to close files that were direct opened through **IORING_OP_OPENAT**, **IORING_OP_OPENAT2**, or **IORING_OP_ACCEPT** using the `io_uring` specific direct descriptors. Note that only one of the descriptor fields may be set. The direct close feature is available since the 5.15 kernel, where direct descriptors were introduced.

Towards Async Everything Part 2

Scopes, Construction, and Destruction



Robert Leahy

It's Scopes All the Way Down

Document Number: P3955R1

Date: 2026-08-09

Reply-to: Robert Leahy <rleahy@rleahy.ca>

Audience: LEWG, SG1

Abstract

This paper lays out an approach to achieving the same effect as regular, synchronous scopes in the asynchronous domain (i.e. `std::execution [1]`), including asynchronous construction and destruction of objects.

Background

Even C has scopes. Unlike C++, however, nothing (except allocating and deallocating stack

It is becoming apparent that all the sender/receiver features are language features being implemented in library.

K. Shoop. `async-object` - aka `async-RAII` P2849R0

```
template<typename Sender>  
concept enter_scope_sender = /* ... */;  
template<typename Sender, typename... Env>  
concept enter_scope_sender_in = /* ... */;
```

```
template<typename Sender>  
concept exit_scope_sender = /* ... */;  
template<typename Sender, typename... Env>  
concept exit_scope_sender_in = /* ... */;
```

```
template<typename Sender, typename... Env>  
using exit_scope_sender_of_t = /* ... */;
```


Reflections on Completion Signatures

Document Number: P4357R0

Date: 2026-08-27

Reply-to: Robert Leahy <rleahy@rleahy.ca>

Audience: SG1, LEWG

Abstract

This paper proposes changes and additions to better integrate `std::meta::info` into the calculation of completion signatures in `std::execution`.

Background

Senders advertise the ways in which they complete via a `get_completion_signatures` static, consteval template member function. This function returns an instance of an instantiation of

```
template<typename Sndr, typename... Env>  
constexpr valid-completion-signatures auto get_completion_signatures();
```

```
template<typename Sndr, typename... Env>  
constexpr valid-completion-signatures auto get_completion_signatures();
```

```
template <class Range = std::initializer_list<std::meta::info>>  
constexpr std::meta::info get_completion_signatures_type(  
    std::meta::info sender,  
    Range&& env = {});
```

```
template<std::execution::sender Sender, enter_scope_sender Scope>  
std::execution::sender auto within(Sender&& sndr, Scope&& scope);
```

```
std::mutex m;  
{  
    const std::lock_guard _(m);  
    // ...  
}
```

```
std::mutex m;  
{  
    const std::lock_guard _(m);  
    // ...  
}
```

```
template<typename Sender>  
concept enter_scope_sender = /* ... */;  
template<typename Sender, typename... Env>  
concept enter_scope_sender_in = /* ... */;
```

```
std::mutex m;  
{  
    const std::lock_guard _(m);  
    // ...  
}
```

```
template<std::execution::sender Sender, enter_scope_sender Scope>  
std::execution::sender auto within(Sender&& sndr, Scope&& scope);
```

```
std::mutex m;  
{  
    const std::lock_guard _(m);  
    // ...  
}
```

```
template<typename Sender>  
concept exit_scope_sender = /* ... */;  
template<typename Sender, typename... Env>  
concept exit_scope_sender_in = /* ... */;
```

```
struct within_t {  
    template <std::execution::sender Sndr, enter_scope_sender Scope>  
    static constexpr std::execution::sender auto operator()(  
        Sndr&& sndr, Scope&& scope) noexcept(  
        std::is_nothrow_constructible_v<std::remove_cvref_t<Sndr>, Sndr> &&  
        std::is_nothrow_constructible_v<std::remove_cvref_t<Scope>, Scope>)  
    {  
        return make_sender(  
            within_t{},  
            {},  
            std::forward<Sndr>(sndr),  
            std::forward<Scope>(scope));  
    }  
};
```

```
template <>
struct impls-for<within_t> : default-impls {

    template <typename Sndr, typename... Env>
    static consteval std::meta::info get-completion-signatures();

    static constexpr auto connect = /* ... */;

};
```

```
template <typename Sender, typename... Env>  
static constexpr std::meta::info get-completion-signatures() {  
  
    const auto body = ^^std::execution::child-type<Sender, 0>;  
    const auto decayed_body = remove_cvref(body);  
    if (!is_constructible_type(decayed_body, {body})) {  
        throw std::meta::exception("The body sender is not decay-copyable.",  
                                    body);  
    }  
  
    // ...  
}
```

```
template <typename Sender, typename... Env>
static constexpr std::meta::info get-completion-signatures() {
    // ...

    const auto scope = ^^std::execution::child-type<Sender, 1>;
    const auto exit_sender = exit_scope_sender_type(scope, {^^Env...});
    check_exit_scope_sender(exit_sender, {^^Env...});

    // ...
}
```

```
template <typename Sender, typename... Env>
static constexpr std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set signatures;
    signatures.add_signatures(
        std::execution::get_stored_completion_signatures_type(
            get-body-completion-signatures-type(decayed_body, {^^Env...})));

    // ...
}
```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set signatures;
    signatures.add_signatures(
        std::execution::get_stored_completion_signatures_type(
            get-body-completion-signatures-type(decayed_body, {^^Env...})));

    // ...
}
```

```
struct completion_signature_set {  
  
    consteval std::meta::info get() const;  
  
    consteval void add_signature(std::meta::info signature);  
  
    template<typename Range = std::initializer_list<std::meta::info>>  
    consteval void add_signature(std::meta::info tag, Range&& args);  
  
    consteval void add_signatures(std::meta::info signatures);  
  
    template<typename Range = std::initializer_list<std::meta::info>>  
    consteval void add_signatures_of(std::meta::info sender, Range&& env = {});  
  
    consteval void add_throw();  
  
};
```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set signatures;
    signatures.add_signatures(
        std::execution::get_stored_completion_signatures_type(
            get-body-completion-signatures-type(decayed_body, {^^Env...})));

    // ...
}
```

Stop the Decay

Document Number: P4288R1

Date: 2026-07-31

Reply-to: Robert Leahy <rleahy@rleahy.ca>

Audience: SG1, LEWG

Abstract

This paper proposes that `std::execution` algorithms pass through by-reference result datums from child operations rather than uniformly decay-copying them. To aid in the accomplishment thereof it also proposes class templates which abstract the necessary completion storage and completion signature transformation. This paper does not propose changing how `std::execution` algorithms store the arguments passed when creating the sender, or connecting the same.

```
template<typename Signatures>
struct storage_for_completion_signatures {

    template<typename Tag, typename... Args>
    constexpr void arrive(Tag tag, Args&&... args) noexcept;

    template<std::execution::receiver Rcvr>
    constexpr bool complete(Rcvr&& rcvr) && noexcept;

};

constexpr std::meta::info get_stored_completion_signatures_type(
    std::meta::info signatures);
```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set signatures;
    signatures.add_signatures(
        std::execution::get_stored_completion_signatures_type(
            get-body-completion-signatures-type(decayed_body, {^^Env...})));

    // ...
}
```

```

template <typename Range = std::initializer_list<std::meta::info>>
constexpr std::meta::info get-body-completion-signatures-type(
    std::meta::info body,
    Range&& env = {})
{
    completion_signature_set signatures;
    signatures.add_signatures_of(body, env);

    if (std::ranges::empty(env) ||
        !is_nothrow_move_constructible_type(body) ||
        !extract<bool>(substitute(
            ^^std::execution::is_nothrow_connectable_in_v,
            std::views::concat(std::views::single(body),
                               std::forward<Range>(env)))))
    {
        signatures.add_throw();
    }

    return signatures.get();
}

```

```
template <typename Range = std::initializer_list<std::meta::info>>
constexpr std::meta::info get-body-completion-signatures-type(
    std::meta::info body,
    Range&& env = {})
{
    completion_signature_set signatures;
    signatures.add_signatures_of(body, env);

    if (std::ranges::empty(env) ||
        !is_nothrow_move_constructible_type(body) ||
        !extract<bool>(substitute(
            ^^std::execution::is_nothrow_connectable_in_v,
            std::views::concat(std::views::single(body),
                               std::forward<Range>(env)))))
    {
        signatures.add_throw();
    }

    return signatures.get();
}
```

Given a type `Env`, if `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed, the former shall be a superset of the latter, with the difference corresponding to error or stopped completion operations.

§33.9.9 [exec.getcompsigs]

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    for (const auto signature : template_arguments_of(
        std::execution::get_completion_signatures_type(scope, {^^Env...})))
    {
        if (!is_same_type(return_type_of(signature),
                           ^^std::execution::set_value_t))
        {
            signatures.add_signature(signature);
        }
    }
    return signatures.get();
}
```

```
static constexpr auto connect =  
    [<typename Sndr, typename Rcvr>(Sndr&& sndr, Rcvr rcvr)  
    noexcept(/* ... */) {  
        auto& [_, _, body, scope] = sndr;  
        return within-state<  
            std::execution::child-type<Sndr, 0>,  
            std::execution::child-type<Sndr, 1>,  
            Rcvr>(  
                std::forward_like<Sndr>(body),  
                std::forward_like<Sndr>(scope),  
                std::move(rcvr));  
    };
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    using operation_state_concept = std::execution::operation_state_tag;

    constexpr void start() & noexcept;

};
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    using body_t      = std::remove_cvref_t<Body>;
    using env_t       = std::execution::env_of_t<Rcvr>;
    using exit_t      = exit_scope_sender_of_t<Scope, env_t>;
    using storage_t = storage_for_completion_signatures<
        typename[:get-body-completion-signatures-type(^^body_t, {^^env_t}):]>;

    struct enter_receiver;
    struct body_receiver;
    struct exit_receiver;

    using enter_op_t = std::execution::connect_result_t<Scope, enter_receiver>;
    using body_op_t  = std::execution::connect_result_t<body_t, body_receiver>;
    using exit_op_t  = std::execution::connect_result_t<exit_t, exit_receiver>;

};
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    struct enter_state {
        enter_op_t op;
        body_t      body;
    };

    struct body_state {
        body_op_t op;
        exit_t    exit;
    };

};
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    Rcvr                                rcvr;
    storage_t                          storage;
    std::variant<enter_state, body_state, exit_op_t> op;

};
```

```

template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    constexpr within-state(Body&& body, Scope&& scope, Rcvr rcvr)
        noexcept(
            std::is_nothrow_constructible_v<body_t, Body> &&
            std::execution::is_nothrow_connectable_in_v<Scope, env_t>)
        : rcvr(std::move(rcvr))
        , op(std::in_place_type<enter_state>,
            emplace-from([&]() {
                return enter_state{
                    std::execution::connect(std::forward<Scope>(scope),
                                            enter_receiver{*this}),
                    std::forward<Body>(body)});
            })))
    {}

};

```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    constexpr void start() & noexcept {
        std::execution::start(std::get<enter_state>(op).op);
    }

};
```

```
struct enter_receiver {  
    using receiver_concept = std::execution::receiver_tag;  
  
    within-state& state;  
  
    template <typename Error>  
    constexpr void set_error(Error&& error) && noexcept {  
        std::execution::set_error(std::move(state.rcvr),  
                                   std::forward<Error>(error));  
    }  
  
    constexpr void set_stopped() && noexcept {  
        std::execution::set_stopped(std::move(state.rcvr));  
    }  
  
    constexpr env_t get_env() const noexcept {  
        return std::execution::get_env(state.rcvr);  
    }  
  
};
```

```

struct enter_receiver {

    constexpr void set_value(exit_t exit) && noexcept {
        auto& state = this->state;
        try {
            auto body = std::move(std::get<enter_state>(state.op).body);
            auto&& body_op = state.op.template emplace<body_state>(
                emplace-from([&]{
                    return body_state{std::execution::connect(std::move(body),
                                                                body_receiver{state}),
                                     std::move(exit)};
                }));
            std::execution::start(body_op.op);
        } catch (...) {
            // ...
        }
    }

};

```

```

struct enter_receiver {

    constexpr void set_value(exit_t exit) && noexcept {
        auto& state = this->state;
        try {
            auto body = std::move(std::get<enter_state>(state.op).body);
            auto&& body_op = state.op.template emplace<body_state>(
                emplace-from([&]{
                    return body_state{std::execution::connect(std::move(body),
                                                                body_receiver{state}),
                                     std::move(exit)};
                }));
            std::execution::start(body_op.op);
        } catch (...) {
            // ...
        }
    }

};

```

Lightning Talk:

Can `set_value` Actually Throw?

ROBERT LEAHY



20
25



```

struct enter_receiver {

    constexpr void set_value(exit_t exit) && noexcept {
        auto& state = this->state;
        try {
            auto body = std::move(std::get<enter_state>(state.op).body);
            auto&& body_op = state.op.template emplace<body_state>(
                emplace-from([&]{
                    return body_state{std::execution::connect(std::move(body),
                                                                body_receiver{state}),
                                     std::move(exit)};
                }));
            std::execution::start(body_op.op);
        } catch (...) {
            // ...
        }
    }

};

```

```

struct enter_receiver {

    constexpr void set_value(exit_t exit) && noexcept {
        auto& state = this->state;
        try {
            auto body = std::move(std::get<enter_state>(state.op).body);
            auto&& body_op = state.op.template emplace<body_state>(
                emplace-from([&]{
                    return body_state{std::execution::connect(std::move(body),
                                                                body_receiver{state}),
                                     std::move(exit)};
                }));
            std::execution::start(body_op.op);
        } catch (...) {
            // ...
        }
    }

};

```

The initializations of the elements of the aggregate are evaluated in the element order. That is, all value computations and side effects associated with a given element are sequenced before those of any element that follows it in order.

§9.5.2 [dcl.init.aggr]

```

struct enter_receiver {

    constexpr void set_value(exit_t exit) && noexcept {
        auto& state = this->state;
        try {
            // ...
        } catch (...) {
            if constexpr (
                !std::is_nothrow_move_constructible_v<body_t> ||
                !std::execution::is_nothrow_connectable_in_v<body_t, env_t>)
            {
                state.storage.arrive(std::execution::set_error,
                                     std::current_exception());
                state.exit(std::move(exit));
            }
        }
    };
};

```

```
struct body_receiver {  
    using receiver_concept = std::execution::receiver_tag;  
    within-state& state;  
    template <typename... Args>  
    constexpr void set_value(Args&&... args) && noexcept {  
        state.body_complete(std::execution::set_value,  
                             std::forward<Args>(args)...);  
    }  
    template <typename Error>  
    constexpr void set_error(Error&& error) && noexcept {  
        state.body_complete(std::execution::set_error,  
                             std::forward<Error>(error));  
    }  
    constexpr void set_stopped() && noexcept {  
        state.body_complete(std::execution::set_stopped);  
    }  
    constexpr env_t get_env() const noexcept {  
        return std::execution::get_env(state.rcvr);  
    }  
};
```

```
struct body_receiver {  
    using receiver_concept = std::execution::receiver_tag;  
    within-state state;  
    template <typename... Args>  
    constexpr void set_value(Args&&... args) && noexcept {  
        state.body_complete(std::execution::set_value,  
                             std::forward<Args>(args)...);  
    }  
    template <typename Error>  
    constexpr void set_error(Error&& error) && noexcept {  
        state.body_complete(std::execution::set_error,  
                             std::forward<Error>(error));  
    }  
    constexpr void set_stopped() && noexcept {  
        state.body_complete(std::execution::set_stopped);  
    }  
    constexpr env_t get_env() const noexcept {  
        return std::execution::get_env(state.rcvr);  
    }  
};
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    template <typename Tag, typename... Args>
    constexpr void body_complete(Tag tag, Args&&... args) noexcept {
        storage.arrive(tag, std::forward<Args>(args)...);
        exit(std::move(std::get<body_state>(op).exit));
    }

};
```

```
template <typename Body, typename Scope, typename Rcvr>
struct within-state {

    constexpr void exit(exit_t exit) noexcept {
        auto&& exit_op = op.template emplace<exit_op_t>(
            emplace-from([&]() noexcept {
                return std::execution::connect(std::move(exit),
                                                exit_receiver{*this});
            }));
        std::execution::start(exit_op);
    }

};
```

```
struct exit_receiver {  
    using receiver_concept = std::execution::receiver_tag;  
  
    within-state& state;  
  
    constexpr void set_value() && noexcept {  
        std::move(state.storage).complete(std::move(state.rcvr));  
    }  
  
    constexpr env_t get_env() const noexcept {  
        return std::execution::get_env(state.rcvr);  
    }  
};
```

Primitives for Non-Local Concurrency

Document #: P4215R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-12
Project: Programming Language C++
Audience: SG1
Reply-to: Lucian Radu Teodorescu (Garmin)
<lucteo@lucteo.ro>

Abstract

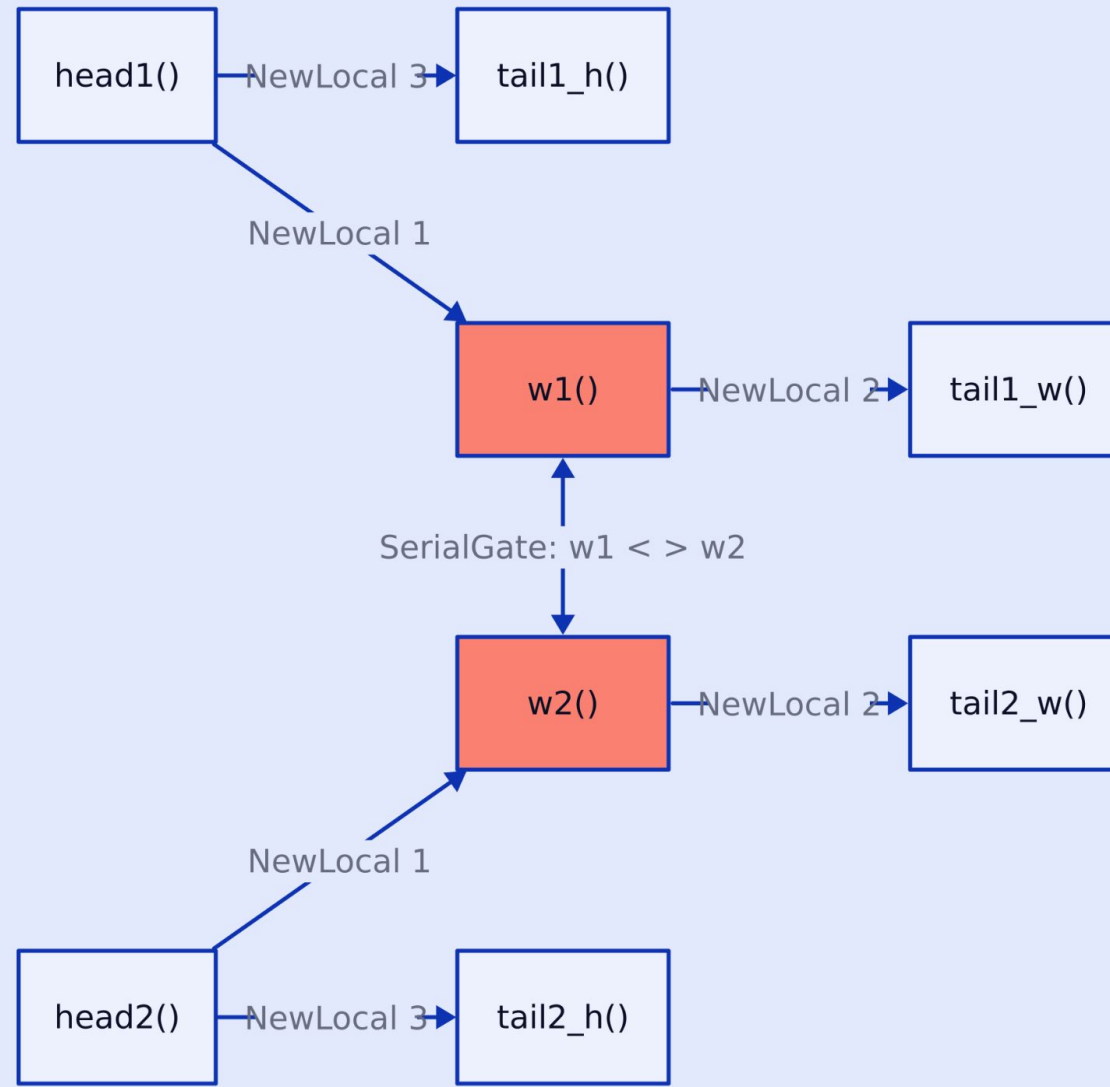
C++26 senders provide a vocabulary for composing asynchronous work, but they do not by themselves provide sender-native equivalents of synchronization primitives that impose non-local concurrency constraints. Programs still need to serialize unrelated operations, bound concurrency across independently submitted work, wait for readiness, and coordinate completion or phase boundaries.

This paper explores a family of sender-based primitives, called *gates*, that control admission of work according to such non-local constraints. The design is motivated by the ordering relations expressed by existing synchronization primitives, but aims to preserve sender composition, safety invariants, progress, non-blocking waiting, and compatibility with structured lifetime management.

The paper is exploratory. It asks SG1 whether this model is a useful direction for C++ concurrency, and where further design effort should be focused.

1 Introduction

serial_gate

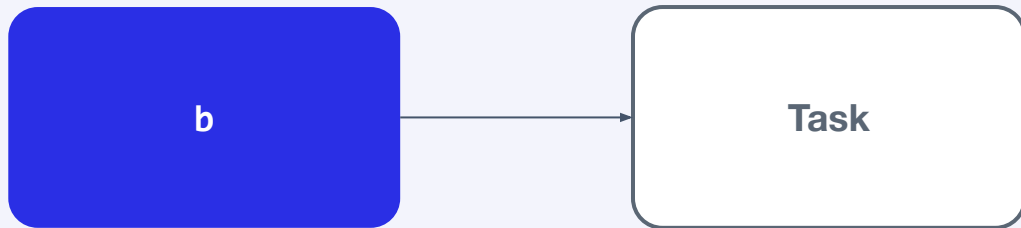
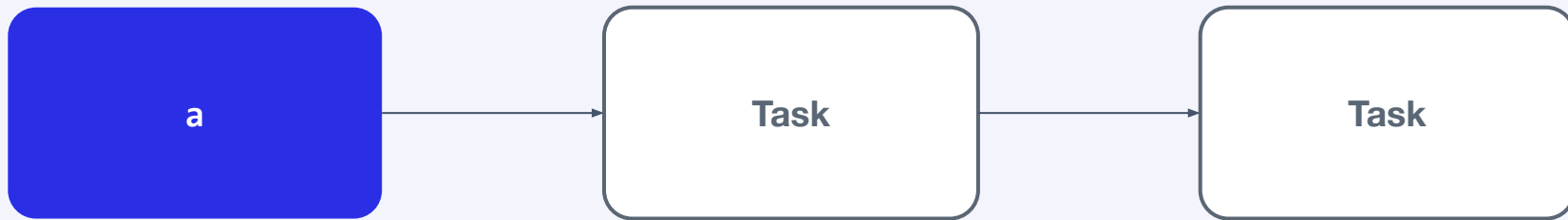


```
std::execution::serial_gate gate;

std::execution::sender auto sndr = std::execution::within(
    std::execution::just() | std::execution::then([]() {
        std::cout << "Within critical section" << std::endl;
    })),
    gate.acquire());
```

```
std::execution::serial_gate a;  
std::execution::serial_gate b;  
  
std::execution::sender auto sndr = std::execution::within(  
    std::execution::within(  
        std::execution::just() | std::execution::then([]() {  
            std::cout << "Within critical section" << std::endl;  
        })),  
        b.acquire()),  
    a.acquire());
```

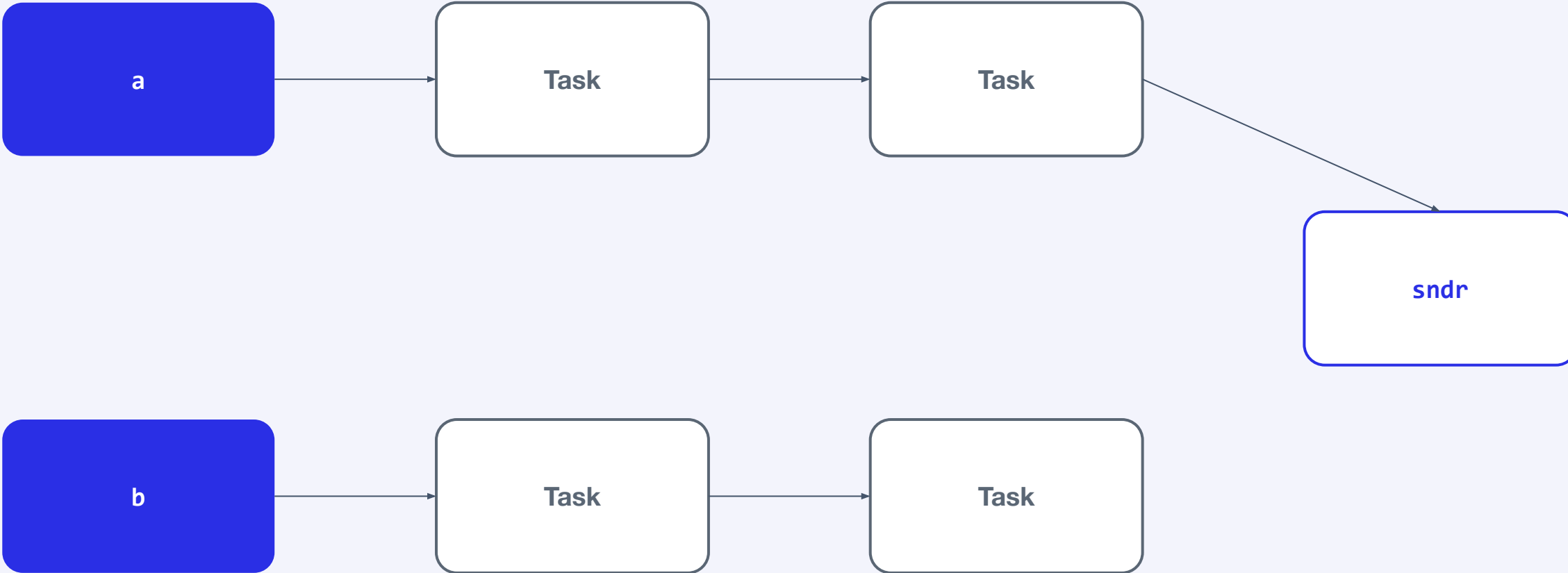
Two serial_gates Entered Serially



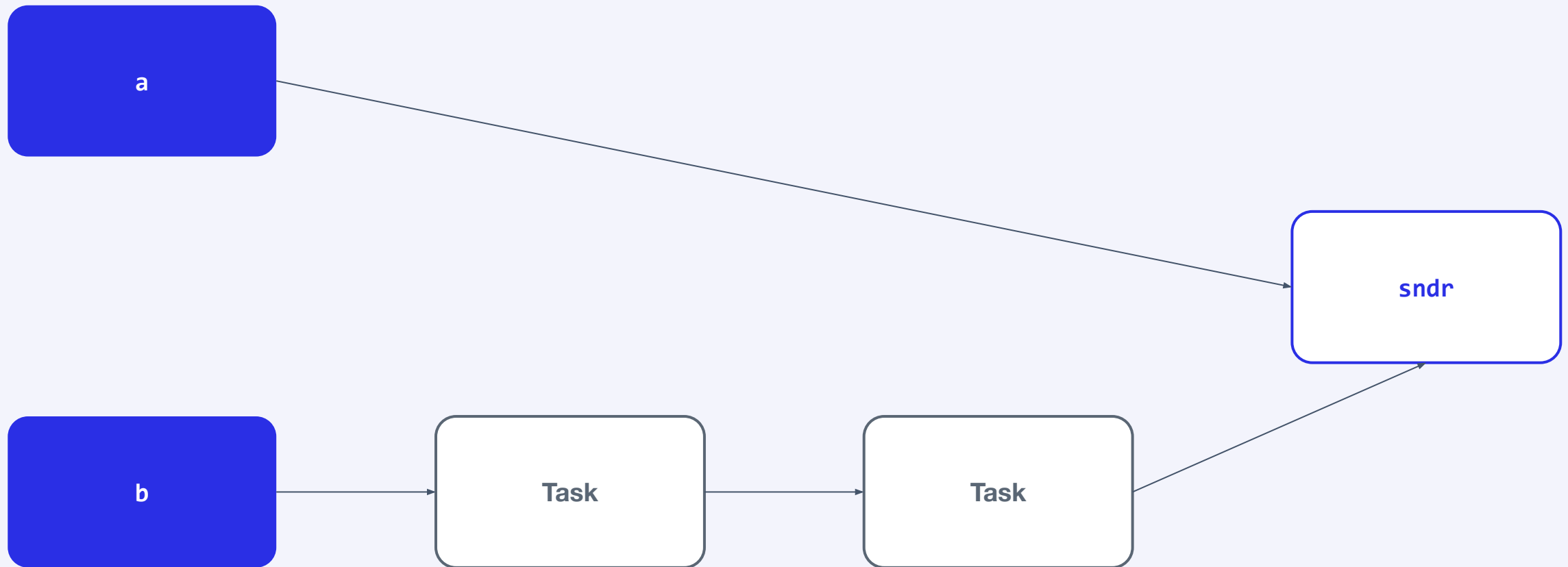
Two serial_gates Entered Serially



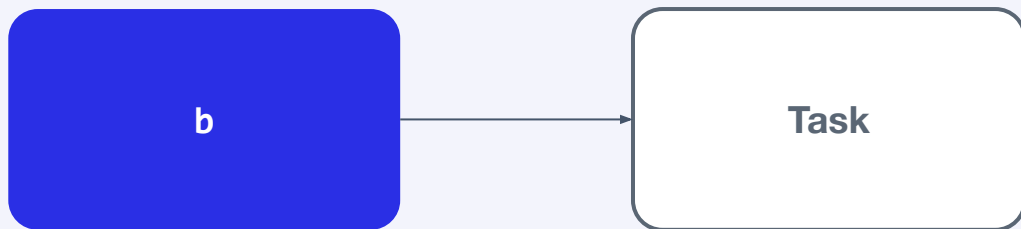
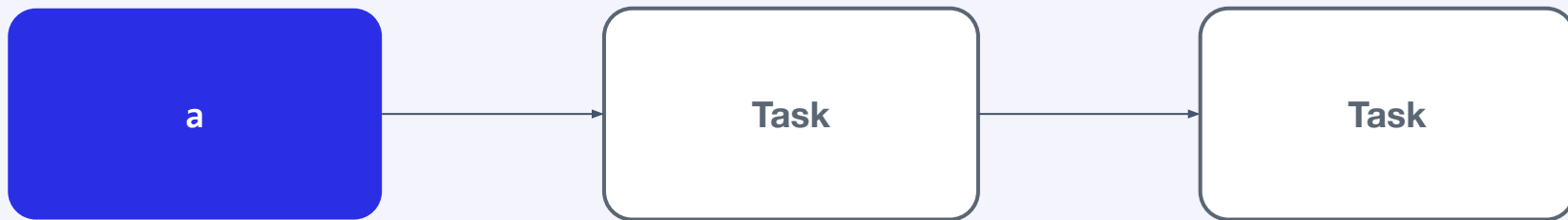
Two serial_gates Entered Serially



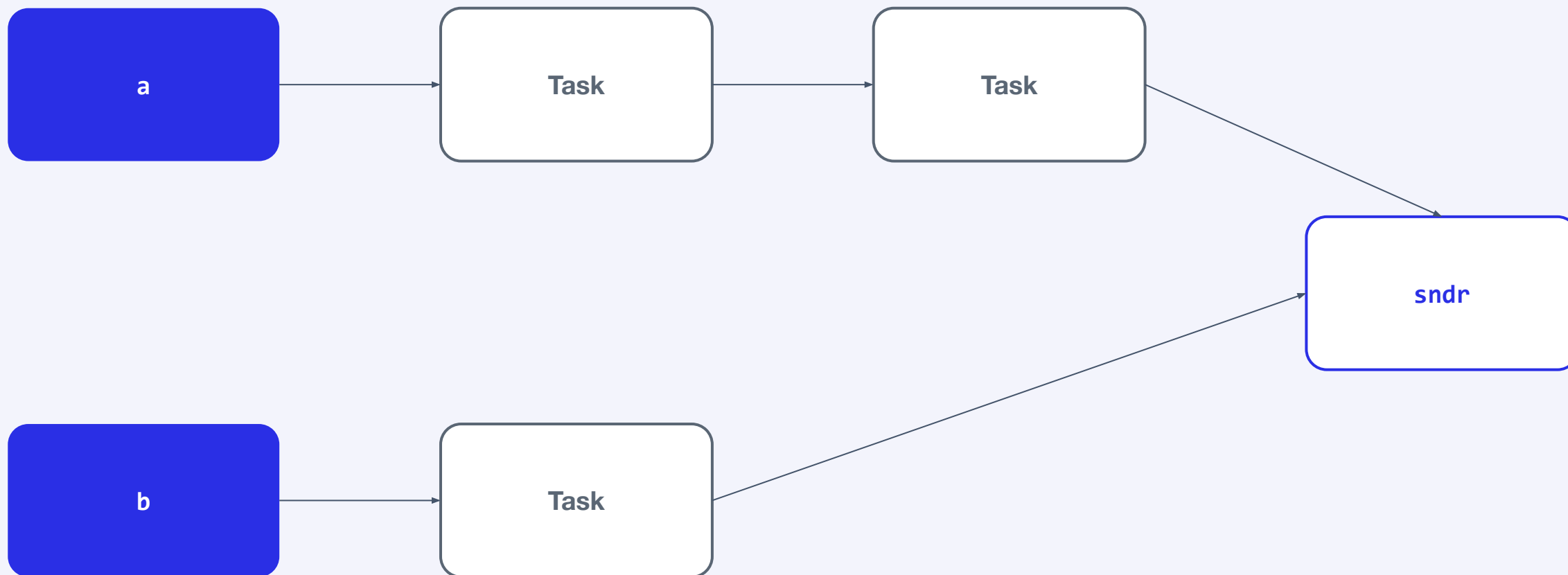
Two serial_gates Entered Serially



Two serial_gates Entered Concurrently



Two serial_gates Entered Concurrently



```
template<enter_scope_sender... Senders>  
enter_scope_sender auto enter_scopes(Senders&&... sndrs);
```

```
std::execution::serial_gate a;  
std::execution::serial_gate b;  
  
std::execution::sender auto sndr = std::execution::within(  
    std::execution::just() | std::execution::then([]() {  
        std::cout << "Within critical section" << std::endl;  
    })),  
    std::execution::enter_scopes(  
        a.acquire(),  
        b.acquire())));
```

```

struct enter_scopes_t {
    template <enter_scope_sender... Scopes>
    static constexpr enter_scope_sender auto operator()(Scopes&&... scopes)
        noexcept((std::is_nothrow_constructible_v<
            std::remove_cvref_t<Scopes>, Scopes> && ...))
    {
        return make_sender(enter_scopes_t{}, {}, std::forward<Scopes>(scopes)...);
    }

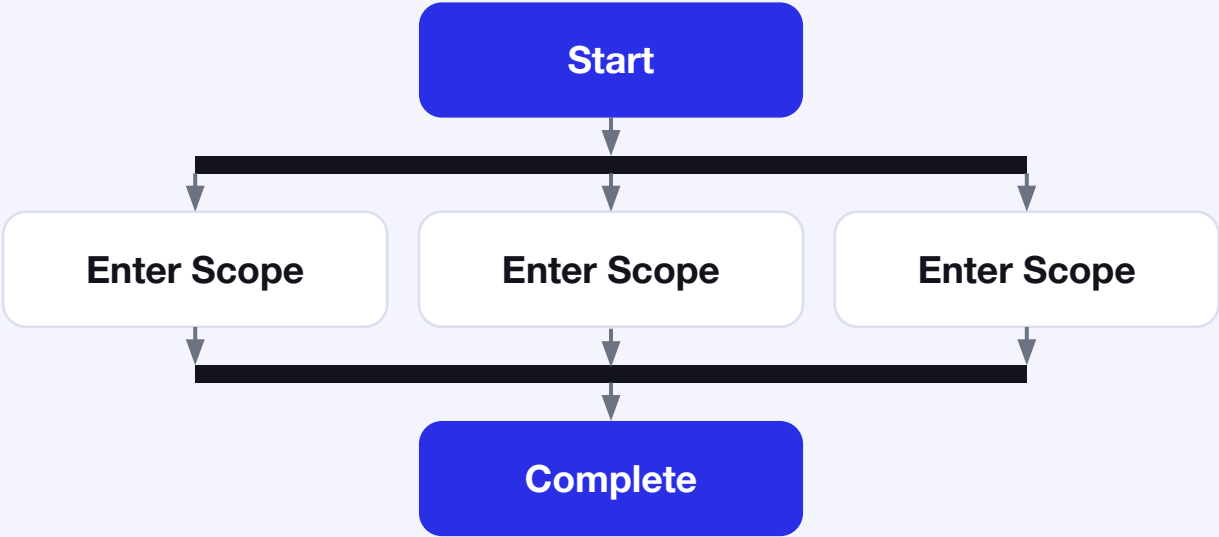
    template <enter_scope_sender Scope>
    static constexpr enter_scope_sender auto operator()(Scope&& scope)
        noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<Scope>, Scope>)
    {
        return std::forward<Scope>(scope);
    }

    static constexpr enter_scope_sender auto operator>() noexcept {
        return std::execution::just(std::execution::just());
    }
};

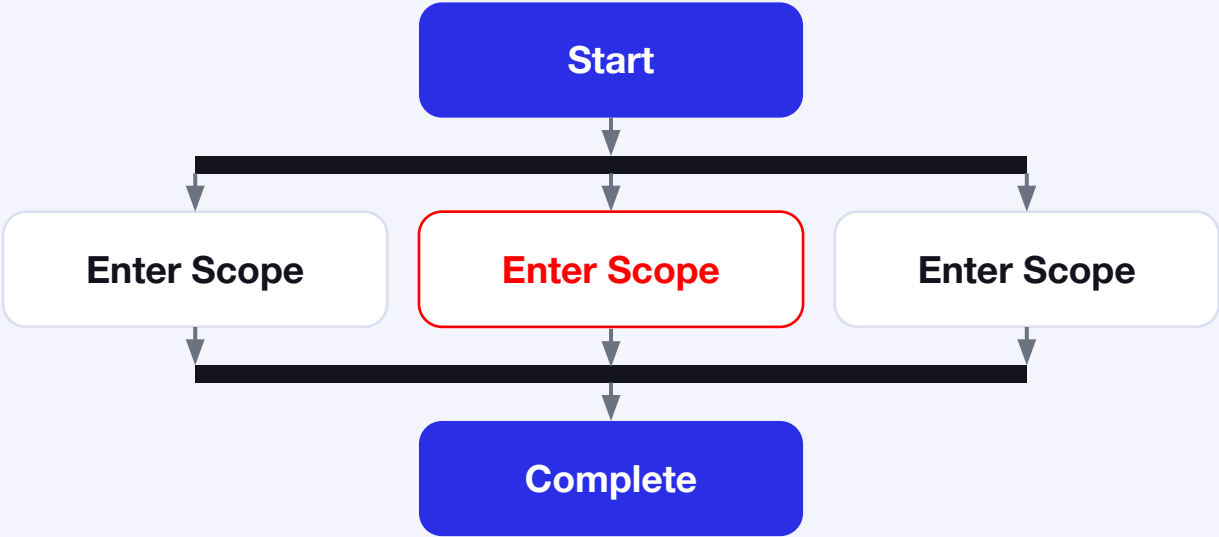
```

```
struct enter_scopes_t {  
    template <enter_scope_sender... Scopes>  
    static constexpr enter_scope_sender auto operator()(Scopes&&... scopes)  
        noexcept((std::is_nothrow_constructible_v<  
            std::remove_cvref_t<Scopes>, Scopes> && ...))  
    {  
        return  
            std::execution::when_all(std::forward<Scopes>(scopes)...) |  
            std::execution::then([](exit_scope_sender auto&&... sndrs) noexcept {  
                return std::execution::when_all(  
                    std::forward<decltype(sndrs)>(sndrs)...);  
                });  
    }  
};
```

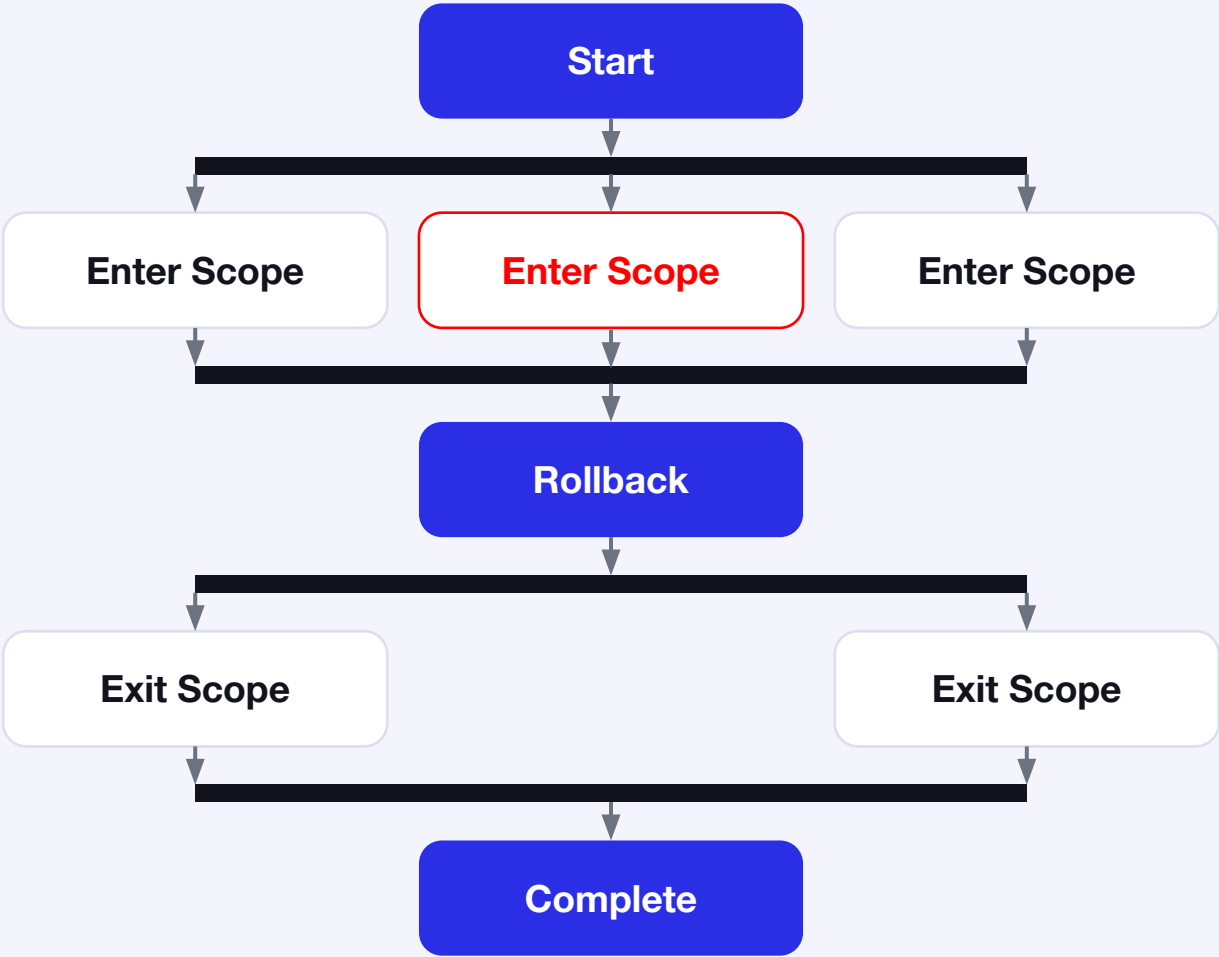
`enter_scopes`



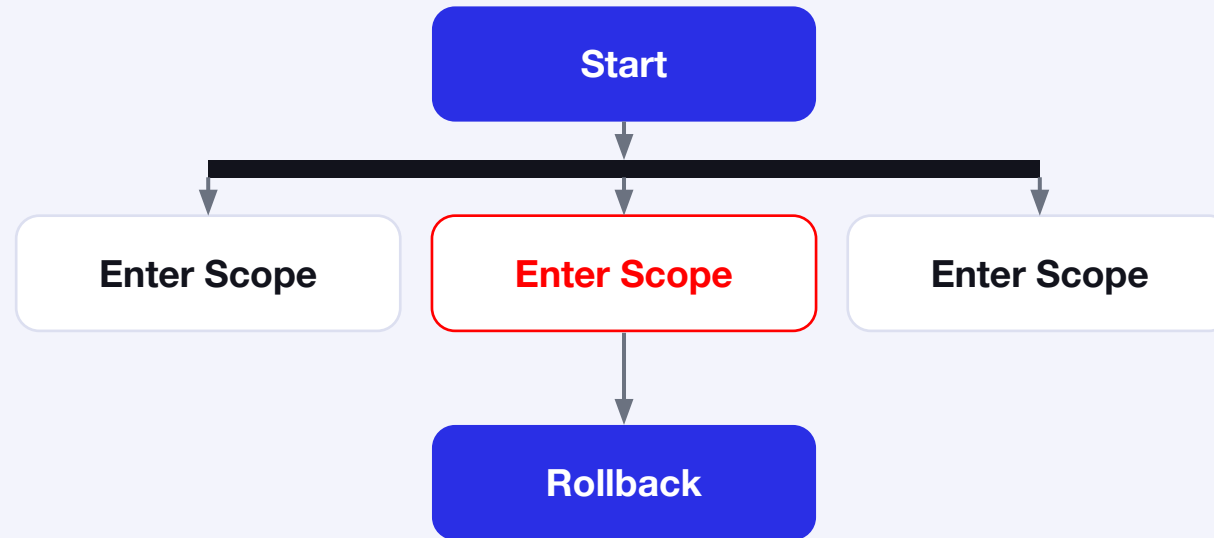
enter_scopes



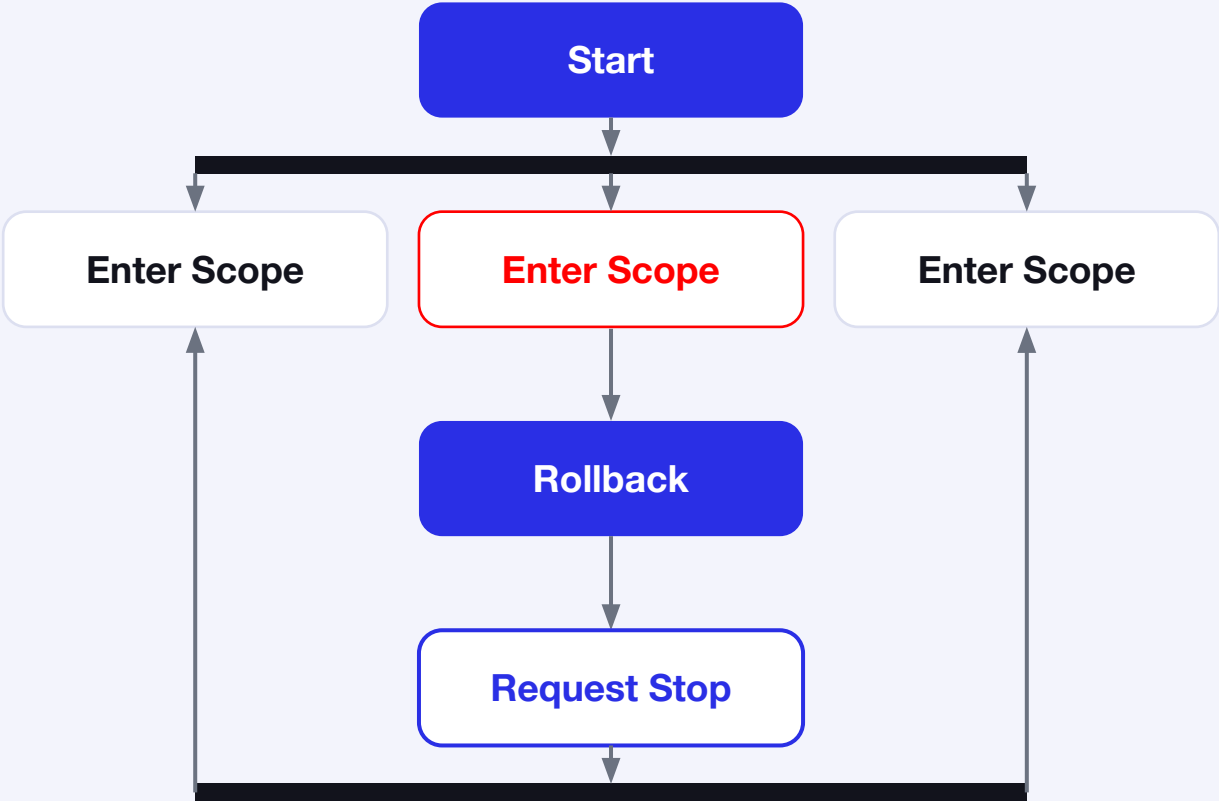
enter_scopes



enter_scopes



enter_scopes



```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {

    constexpr auto indices = std::views::iota(
        std::size_t{0},
        std::execution::indices-for<Sender>::size());

    bool has_unhappy_completions = false;
    template for (constexpr auto i : indices) {
        const auto child = ^^std::execution::child-type<Sender, i>;
        if (can-complete-unhappily(child, {^^Env...})) {
            has_unhappy_completions = true;
        }
    }

    // ...
}
```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {

    constexpr auto indices = std::views::iota(
        std::size_t{0},
        std::execution::indices-for<Sender>::size());

    bool has_unhappy_completions = false;
    template for (constexpr auto i : indices) {
        const auto child = ^^std::execution::child-type<Sender, i>;
        if (can-complete-unhappily(child, {^^Env...})) {
            has_unhappy_completions = true;
        }
    }

    // ...
}
```

```
template <typename Range = std::initializer_list<std::meta::info>>
constexpr bool can-complete-unhappily(
    std::meta::info sender,
    Range&& env = {})
{
    return std::ranges::any_of(
        template_arguments_of(
            std::execution::get_completion_signatures_type(
                sender,
                std::forward<Range>(env))),
        [](std::meta::info signature) {
            return !is_same_type(
                return_type_of(signature),
                ^^std::execution::set_value_t);
        });
}
```

```
template <typename Sender, typename... Env>
static constexpr std::meta::info get-completion-signatures() {
    // ...

    std::vector<std::meta::info> enter_env{^^Env...};
    if constexpr (sizeof...(Env) == 1) {
        if (has_unhappy_completions) {
            enter_env = std::vector<std::meta::info>{
                substitute(^^child-env-t, {^^Env...})};
        }
    }

    // ...
}
```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    std::vector<std::meta::info> enter_env{^^Env...};
    if constexpr (sizeof...(Env) == 1) {
        if (has_unhappy_completions) {
            enter_env = std::vector<std::meta::info>{
                substitute(^^child-env-t, {^^Env...})});
        }
    }

    // ...
}
```

```
template <typename Env>
using child-env-t = std::execution::env<std::execution::prop<std::get_stop_token_t,
                                                             std::inplace_stop_token>,
                                         Env>;
```

```

template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set forwarded_signatures;
    std::vector<std::meta::info> exit_senders;
    template for (constexpr auto i : indices) {
        const auto child = ^^std::execution::child-type<Sender, i>;

        forwarded_signatures.add_signatures(
            get-forwarded-completion-signatures-type({
                std::execution::get_completion_signatures_type(child, enter_env)}));

        const auto exit_sender = exit_scope_sender_type(child, enter_env);
        check_exit_scope_sender(exit_sender, {^^Env...});
        exit_senders.push_back(exit_sender);
    }

    // ...
}

```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set forwarded_signatures;
    std::vector<std::meta::info> exit_senders;
    template for (constexpr auto i : indices) {
        const auto child = ^^std::execution::child-type<Sender, i>;

        forwarded_signatures.add_signatures(
            get-forwarded-completion-signatures-type({
                std::execution::get_completion_signatures_type(child, enter_env)}));

        const auto exit_sender = exit_scope_sender_type(child, enter_env);
        check_exit_scope_sender(exit_sender, {^^Env...});
        exit_senders.push_back(exit_sender);
    }

    // ...
}
```

```

template <typename Range = std::initializer_list<std::meta::info>>
constexpr std::meta::info get-forwarded-completion-signatures-type(
    Range&& input_signatures)
{
    completion_signature_set signatures;
    for (const auto input : input_signatures) {
        for (const auto signature : template_arguments_of(input)) {
            if (!is_same_type(return_type_of(signature),
                              ^^std::execution::set_value_t))
            {
                signatures.add_signature(signature);
            }
        }
    }
    return signatures.get();
}

```

```

template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set forwarded_signatures;
    std::vector<std::meta::info> exit_senders;
    template for (constexpr auto i : indices) {
        const auto child = ^^std::execution::child-type<Sender, i>;

        forwarded_signatures.add_signatures(
            get-forwarded-completion-signatures-type({
                std::execution::get_completion_signatures_type(child, enter_env)}));

        const auto exit_sender = exit_scope_sender_type(child, enter_env);
        check_exit_scope_sender(exit_sender, {^^Env...});
        exit_senders.push_back(exit_sender);
    }

    // ...
}

```

```
template <typename Sender, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    // ...

    completion_signature_set signatures;
    signatures.add_signatures(
        std::execution::get_stored_completion_signatures_type(
            forwarded_signatures.get()));

    const auto exit_type = invoke_result(^std::execution::when_all_t,
                                         exit_senders);
    signatures.add_signature(^std::execution::set_value_t, {exit_type});

    return signatures.get();
}
```

```
static constexpr auto connect =  
    [<typename Sndr, typename Rcvr>(Sndr&& sndr, Rcvr rcvr)  
    noexcept(/* ... */) {  
        auto& [_, _, ...scopes] = sndr;  
        return enter-scopes-state<  
            Rcvr,  
            std::execution::indices-for<Sndr>,  
            decltype(std::forward_like<Sndr>(scopes))...>(  
                std::move(rcvr),  
                std::forward_like<Sndr>(scopes)...);  
    };
```

```
template <typename Rcvr, typename Indices, typename... Scopes>
struct enter-scopes-state;

template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {
    // ...
};
```

```
template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {

    using env_t = std::execution::env_of_t<Rcvr>;

    static constexpr bool has_unhappy_completions =
        (can-complete-unhappily(^^Scopes, {^^env_t}) || ...);

    using enter_env_t = std::conditional_t<has_unhappy_completions,
                                           child-env-t<env_t>,
                                           env_t>;

    template <std::size_t I>
    struct enter_receiver;

    struct rollback_receiver;

};
```

```

template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {
    struct enter_state;
    struct rollback_state;

    std::tuple<
        std::optional<
            exit_scope_sender_of_t<Scopes, enter_env_t>>...> exit_senders;

    std::execution::storage_for_completion_signatures<
        typename[:get-forwarded-completion-signatures-type({
            std::execution::get_completion_signatures_type(
                ^^Scopes, {^^enter_env_t})...}):]> storage;

    Rcvr rcvr;
    std::inplace_stop_source stop_source;
    std::atomic<std::size_t> remaining{sizeof...(Scopes)};
    std::variant<enter_state, rollback_state> op;

};

```

```

template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {

    explicit constexpr enter-scopes-state(Rcvr rcvr, Scopes&&... scopes)
        noexcept(std::is_nothrow_constructible_v<enter_state,
                                                    enter-scopes-state&,
                                                    Scopes...>)

        : rcvr(std::move(rcvr))
        , op(std::in_place_type<enter_state>,
            *this,
            std::forward<Scopes>(scopes)...)
    {}

};

```

```

struct enter_state {

    std::optional<std::stop_callback_for_t<std::stop_token_of_t<env_t>,
                                     on-stop-request> on_stop;

    std::atomic<bool> completed_unhappily;
    std::tuple<std::execution::connect_result_t<Scopes,
                                     enter_receiver<Is>>...> ops;

    constexpr explicit enter_state(enter-scopes-state& state,
                                   Scopes&&... scopes) noexcept(
        (std::execution::is_nothrow_connectable_in_v<Scopes,
                                     enter_env_t> && ...))
    : ops(emplace-from([&]() {
        return std::execution::connect(std::forward<Scopes>(scopes),
                                       enter_receiver<Is>{state});
    })...))
    {}

};

```

```
template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {

    constexpr void start() & noexcept {
        auto& enter = std::get<enter_state>(op);

        enter.on_stop.emplace(
            std::get_stop_token(std::execution::get_env(rcvr)),
            on-stop-request{stop_source});

        template for (auto& child : enter.ops) {
            std::execution::start(child);
        }
    }

};
```

```
template <std::size_t I>
struct enter_receiver {
    using receiver_concept = std::execution::receiver_tag;

    enter-scopes-state& state;

    constexpr enter_env_t get_env() const noexcept {
        if constexpr (has_unhappy_completions) {
            return enter_env_t{std::execution::prop{std::get_stop_token,
                                                    state.stop_source.get_token()},
                               std::execution::get_env(state.rcvr)};
        } else {
            return std::execution::get_env(state.rcvr);
        }
    }
};
```

```
template <std::size_t I>
struct enter_receiver {

    template <typename Error>
    constexpr void set_error(Error&& error) && noexcept {
        state.enter_unhappy(std::execution::set_error,
                            std::forward<Error>(error));
    }

    constexpr void set_stopped() && noexcept {
        state.enter_unhappy(std::execution::set_stopped);
    }

    template<exit_scope_sender Exit>
    constexpr void set_value(Exit exit) && noexcept {
        std::get<I>(state.exit_senders).emplace(std::move(exit));
        state.enter_arrive();
    }

};
```

```
template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {

    template <typename Tag, typename... Args>
    constexpr void enter_unhappy(Tag tag, Args&&... args) noexcept {
        auto& enter = std::get<enter_state>(op);

        if (!enter.completed_unhappily.exchange(true,
                                                    std::memory_order_relaxed))
        {
            storage.arrive(tag, std::forward<Args>(args)...);
            stop_source.request_stop();
        }

        enter_arrive();
    }
};
```

```

template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {
    constexpr void enter_arrive() noexcept {
        if (remaining.fetch_sub(1, std::memory_order_release) != 1) {
            return;
        }
        std::atomic_thread_fence(std::memory_order_acquire);

        auto& enter = std::get<enter_state>(op);
        enter.on_stop.reset();

        if (enter.completed_unhappily.load(std::memory_order_relaxed)) {
            rollback();
        } else {
            std::execution::set_value(
                std::move(rcvr),
                std::execution::when_all(
                    std::move(*std::get<Is>(exit_senders))...));
        }
    }
};

```

```
template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter_scopes_state<Rcvr, std::index_sequence<Is...>, Scopes...> {
    constexpr void enter_arrive() noexcept {
        if (remaining.fetch_sub(1, std::memory_order_release) != 1) {
            return;
        }
        std::atomic_thread_fence(std::memory_order_acquire);

        auto& enter = std::get<enter_state>(op);
        enter.on_stop.reset();

        if (enter.completed_unhappily.load(std::memory_order_relaxed)) {
            rollback();
        } else {
            std::execution::set_value(
                std::move(rcvr),
                std::execution::when_all(
                    std::move(*std::get<Is>(exit_senders))...));
        }
    }
};
```

What Are We Synchronizing?

ROBERT LEAHY

```
template <typename Rcvr, std::size_t... Is, typename... Scopes>
struct enter-scopes-state<Rcvr, std::index_sequence<Is...>, Scopes...> {

    constexpr void rollback() noexcept {
        auto& rollback = op.template emplace<rollback_state>(*this);
        if (remaining.load(std::memory_order_relaxed) == 0) {
            std::move(storage).complete(std::move(rcvr));
        } else {
            rollback.start();
        }
    }
};
```

```
struct rollback_state {  
  
    std::tuple<  
        std::optional<  
            std::execution::connect_result_t<  
                exit_scope_sender_of_t<Scopes, enter_env_t>,  
                rollback_receiver>>...> ops;  
  
};
```

```

struct rollback_state {

    constexpr explicit rollback_state(enter-scopes-state& state) noexcept {
        template for (constexpr auto i : std::views::iota(std::size_t{0},
                                                            sizeof...(Scopes)))
        {
            auto& exit = std::get<i>(state.exit_senders);
            if (exit) {
                state.remaining.fetch_add(1, std::memory_order_relaxed);
                std::get<i>(ops).emplace(
                    emplace-from([&]() noexcept {
                        return std::execution::connect(
                            std::move(*exit),
                            rollback_receiver{state});
                    }));
            }
        }
    }

};

```

```
struct rollback_state {  
  
    constexpr void start() & noexcept {  
        template for (auto& op : ops) {  
            if (op) {  
                std::execution::start(*op);  
            }  
        }  
    }  
};
```

```
struct rollback_receiver {  
    using receiver_concept = std::execution::receiver_tag;  
  
    enter-scopes-state& state;  
  
    constexpr void set_value() && noexcept {  
        if (state.remaining.fetch_sub(1, std::memory_order_release) == 1) {  
            std::atomic_thread_fence(std::memory_order_acquire);  
            std::move(state.storage).complete(std::move(state.rcvr));  
        }  
    }  
  
    constexpr env_t get_env() const noexcept {  
        return std::execution::get_env(state.rcvr);  
    }  
};
```

```
file f("foo.txt");  
std::vector<std::byte> buffer{1024};  
const auto read = f.read_some(buffer);  
// ...
```

```
template<typename Object>
concept object = /* ... */;
template<typename Object, typename... Env>
concept object_in = /* ... */;

template<typename Object>
using type_of_object_t = /* ... */;

template<typename Object>
using enter_scope_sender_of_object_t = /* ... */;

constexpr std::meta::info type_of_object_type(std::meta::info object);

constexpr std::meta::info enter_scope_sender_of_object_type(
    std::meta::info object);
```

```
template<typename F, object... Objects>  
std::execution::sender auto lifetime(F&& f, Objects&&... objs);
```

```
std::execution::sender auto sndr = std::execution::lifetime(
    [](
        file::type& f,
        std::vector<std::byte>& buffer)
    {
        return
            f.read_some(buffer) |
            std::execution::then([&](const std::size_t read) {
                // ...
            });
    },
    file("foo.txt"),
    std::execution::make_sync_object<std::vector<std::byte>>(1024));
```

```
within(  
  std::execution::just() | std::execution::let_value(...),  
  enter_scopes(  
    std::invoke(  
      std::forward<Objects>(objects),  
      ...))  
  )  
)
```

```

struct lifetime_t {
    template<typename F, object... Objects>
    static constexpr std::execution::sender auto operator()(
        F&& f,
        Objects&&... objects) noexcept(
            std::is_nothrow_constructible_v<
                std::decay_t<F>,
                F> &&
            (std::is_nothrow_constructible_v<
                std::remove_cvref_t<Objects>,
                Objects> && ...))
    {
        return make_sender(
            lifetime_t{},
            std::tuple(
                std::forward<F>(f),
                std::forward<Objects>(objects)...));
    }
};

```

```

template<typename Self, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    const auto data = ^^std::execution::data-type<Self>;
    const auto size = template_arguments_of(remove_cvref(data)).size();
    const auto get = [&](const std::size_t i) {
        return dealias(substitute(
            ^^get-t,
            {data, std::meta::reflect_constant(i)}));
    };

    std::vector<std::meta::info> objects;
    for (std::size_t i = 1; i != size; ++i) {
        objects.push_back(get(i));
    }

    return std::execution::get_completion_signatures_type(
        get-within-sender-type(get(0), objects),
        {^^Env...});
}

```

```

template<typename Self, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    const auto data = ^^std::execution::data-type<Self>;
    const auto size = template_arguments_of(remove_cvref(data)).size();
    const auto get = [&](const std::size_t i) {
        return dealias(substitute(
            ^^get-t,
            {data, std::meta::reflect_constant(i)}));
    };

    std::vector<std::meta::info> objects;
    for (std::size_t i = 1; i != size; ++i) {
        objects.push_back(get(i));
    }

    return std::execution::get_completion_signatures_type(
        get-within-sender-type(get(0), objects),
        {^^Env...});
}

```

```
template<typename T, std::size_t I>  
using get-t = decltype(std::get<I>(std::declval<T>()));
```

```

template<typename Self, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    const auto data = ^^std::execution::data-type<Self>;
    const auto size = template_arguments_of(remove_cvref(data)).size();
    const auto get = [&](const std::size_t i) {
        return dealias(substitute(
            ^^get-t,
            {data, std::meta::reflect_constant(i)}));
    };

    std::vector<std::meta::info> objects;
    for (std::size_t i = 1; i != size; ++i) {
        objects.push_back(get(i));
    }

    return std::execution::get_completion_signatures_type(
        get-within-sender-type(get(0), objects),
        {^^Env...});
}

```

```

template<typename Self, typename... Env>
static consteval std::meta::info get-completion-signatures() {
    const auto data = ^^std::execution::data-type<Self>;
    const auto size = template_arguments_of(remove_cvref(data)).size();
    const auto get = [&](const std::size_t i) {
        return dealias(substitute(
            ^^get-t,
            {data, std::meta::reflect_constant(i)}));
    };

    std::vector<std::meta::info> objects;
    for (std::size_t i = 1; i != size; ++i) {
        objects.push_back(get(i));
    }

    return std::execution::get_completion_signatures_type(
        get-within-sender-type(get(0), objects),
        {^^Env...});
}

```

```
template<typename Range = std::initializer_list<std::meta::info>>
constexpr std::meta::info get-within-sender-type(
    std::meta::info f,
    Range&& objects = {})
{

    const auto enter_sender = invoke_result(
        ^^enter_scopes_t,
        objects | std::views::transform(enter_scope_sender_of_object_type));

    const auto body_sender = dealias(
        substitute(^^body-sender-t,
            std::views::concat(std::views::single(remove_cvref(f)),
                                objects)));

    return invoke_result(^^within_t, {body_sender, enter_sender});
}
```

```
template<typename Range = std::initializer_list<std::meta::info>>
constexpr std::meta::info get-within-sender-type(
    std::meta::info f,
    Range&& objects = {})
{

    const auto enter_sender = invoke_result(
        ^^enter_scopes_t,
        objects | std::views::transform(enter_scope_sender_of_object_type));

    const auto body_sender = dealias(
        substitute(^^body-sender-t,
            std::views::concat(std::views::single(remove_cvref(f)),
                                objects)));

    return invoke_result(^^within_t, {body_sender, enter_sender});
}
```

```
template<typename F, typename... Objects>
using body-sender-t = decltype(
    std::execution::just() | std::execution::let_value(
        std::declval<body-wrapper<F, Objects...>>()));
```

```
template<typename F, typename... Objects>
using body-sender-t = decltype(
    std::execution::just() | std::execution::let_value(
        std::declval<body-wrapper<F, Objects...>>()));
```

```
template<typename F, typename... Objects>
struct body-wrapper {

    F f;
    storage-for-objects-t<Objects...>& storage;

    constexpr std::invoke_result_t<F, type_of_object_t<Objects>&...>
        operator()() && noexcept(
            std::is_nothrow_invocable_v<F, type_of_object_t<Objects>&...>)
    {
        auto& [...slots] = storage;
        return std::invoke(std::move(f), slots.object...);
    }

};
```

```
template<typename F, typename... Objects>
struct body-wrapper {

    F f;
    storage-for-objects-t<Objects...>& storage;

    constexpr std::invoke_result_t<F, type_of_object_t<Objects>&...>
        operator>()() && noexcept(
            std::is_nothrow_invocable_v<F, type_of_object_t<Objects>&...>)
    {
        auto& [...slots] = storage;
        return std::invoke(std::move(f), slots.object...);
    }

};
```

```
template<typename T>  
union storage-for-object {  
    T object;  
};
```

```
template<typename... Objects>  
using storage-for-objects-t =  
    std::tuple<storage-for-object<type_of_object_t<Object>>...>;
```

```
static constexpr auto connect =  
    []<typename Sndr, typename Rcvr>(Sndr&& sndr, Rcvr rcvr)  
    noexcept(/* ... */)   
    {  
        auto& [_, data] = sndr;  
        auto& [f, ...objects] = data;  
        return lifetime-state<  
            Rcvr,  
            std::remove_cvref_t<decltype(f)>,  
            decltype(std::forward_like<Sndr>(objects))...>(  
                std::move(rcvr),  
                std::forward_like<Sndr>(f),  
                std::forward_like<Sndr>(objects)...);  
    };
```

```
template<typename Rcvr, typename F, typename... Objects>
struct lifetime-state {

    using operation_state_concept = std::execution::operation_state_tag;

    storage-for-objects-t<Objects...> storage;
    std::execution::connect_result_t<
        typename[:get-within-sender-type(^^F, {^^Objects...}):],
        Rcvr> op;

    constexpr void start() & noexcept {
        std::execution::start(op);
    }

};
```

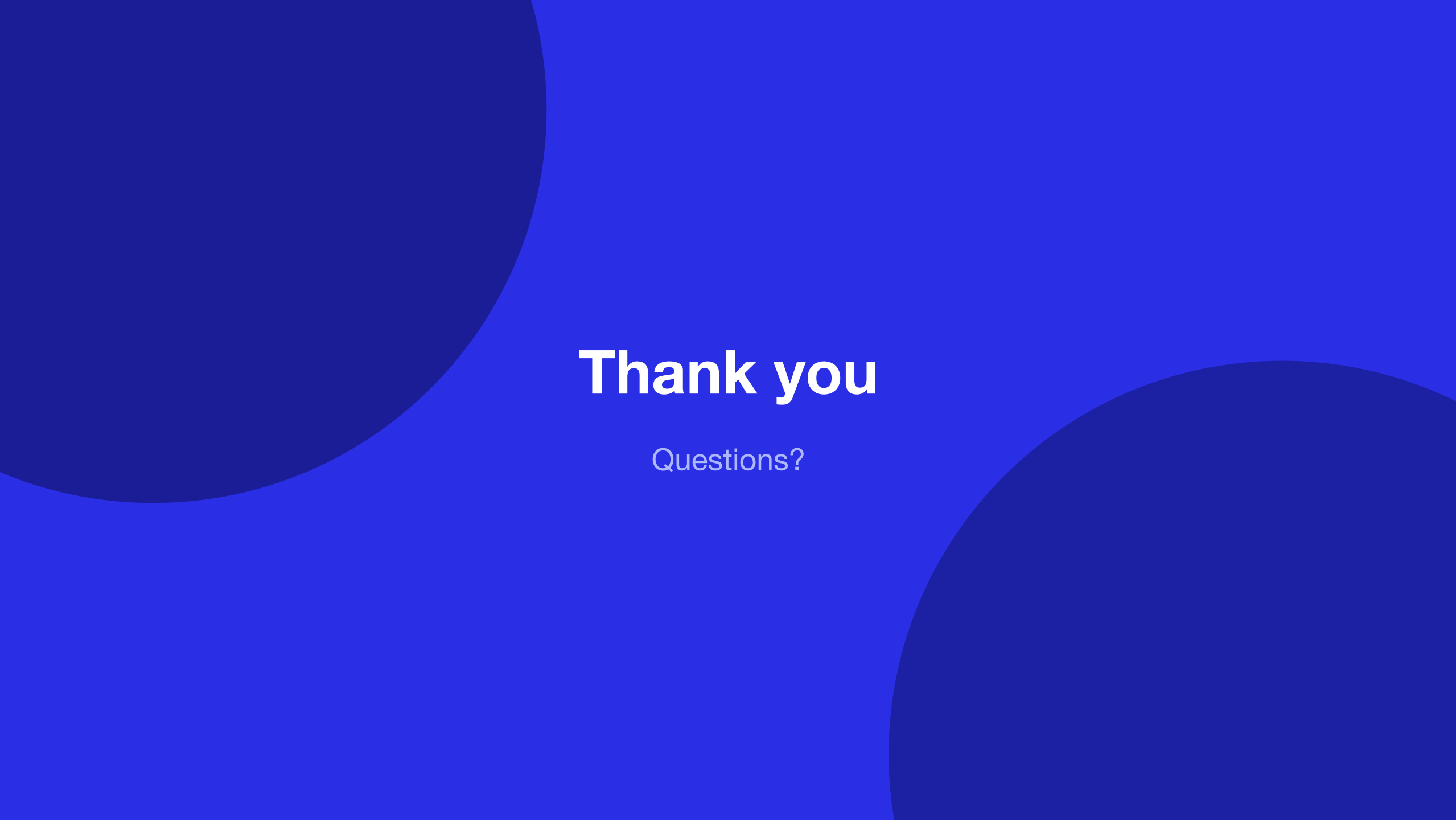
```

template<typename Rcvr, typename F, typename... Objects>
struct lifetime-state {
    constexpr lifetime-state(
        Rcvr rcvr,
        F f,
        Objects&&... objects) noexcept(/* ... */)
    : op([&] {
        auto& [...slots] = storage;
        return std::execution::connect(
            within(
                std::execution::just() | std::execution::let_value(
                    body-wrapper<F, Objects...>{std::move(f), storage}),
                enter_scopes(
                    std::invoke(
                        std::forward<Objects>(objects),
                        std::addressof(slots.object))...)),
            std::move(rcvr));
        }())
    {}
};

```

Conclusion

- 01** RAI is fundamentally about scopes, not objects.
- 02** Enter and exit scope senders project scope boundaries into the asynchronous domain.
- 03** `within` and `enter_scopes` compose scopes.
- 04** An object is a scope plus storage.
- 05** `lifetime` gives asynchronous operations their own fundamentally-asynchronous local variables.



Thank you

Questions?